Codex
装上飞书任务群：一个任务一个群，干完归档散伙（已开源）

> 发布渠道：飞书开发者社区。仓库 https://github.com/OldManWalk/lark-codex-tasks（审完再发）。原始稿同时存于 N100 ~/articles-lct/feishu-dev-community.md。

从两个真实场景说起

场景一：下班交接。 下午五点半，笔记本上的任务跑到一半，我要赶地铁。把项目推给家里常开机的服务器，飞书私聊机器人一句"接着 HANDOFF.md 把 XX 做完"——它建群继续跑。地铁上我点审批卡片，到家时结果卡已经在群里了。

场景二：应急派活。 上周我在外面，临时要交一份周报 PPT。手机飞书私聊一句"帮我把这周的工作记录汇总成周报 PPT 发我"，机器人秒建群「 周报 PPT 制作・a1b2c3」，群里的 Codex 开始翻知识库、汇总记录、生成 PPT。我随手补了一段富文本说明，它排队续接原会话继续执行。

两个场景干完都是同一个动作：我发 /done，结论自动归档成 Markdown（git commit 进知识库），群解散。全程没碰终端。这套系统我把它开源了：[lark-codex-tasks](https://github.com/OldManWalk/lark-codex-tasks)（MIT）。

它和"消息转发桥"不是一回事

社区里已经有不少优秀的飞书 ↔ coding CLI 桥接项目（zarazhangrui/lark-coding-agent-bridge、xile611/lark-codex-bridge 等，它们解决了"消息互通"）。但用下来我发现真正的痛点在工作流层：

- 多个任务挤在一个会话里，上下文互相污染
- 任务跑到一半要审批，人不在电脑前就卡死
- 中途追加需求，要么打断当前执行，要么自己记着待会再说
- 干完的任务没有沉淀，下次想续接翻不到上下文

lark-codex-tasks 的设计答案是：一个任务 = 一个飞书群 = 一条 Codex 线程，配完整的生命周期。

核心工作流

私聊派活（自然语言或 /group）
  ｜ 私聊里的 Codex 只做"分流"：JSON Schema 判定 reply / create_group / resume_task
  ▼
自动建群「 主题 ・短id」，群描述写入任务目标
  ▼
群内执行（auto 模式：日常直接干，特殊操作弹审批卡片）
  ｜ 审批卡片 [☒通过/记住/☒拒绝/☒取消]，回调校验 owner，无超时
  ▼
群内发消息 = 给原任务线程排队派后续活
  ｜ FIFO 持久队列，前序结束自动接续，text 和富文本 post 都受理
  ▼
/done → 归档 Markdown 到知识库（自动 git commit）→ 解散群 → 清注册表

架构上是一条长链：飞书长连接（im.message.receive_v1 + card.action.trigger 双消费者，无需公网 URL，NAT 后的家庭服务器直接跑）→ lark-cli → bridge（Node，零依赖单文件）→ codex app-server JSON-RPC stdio → Codex 线程。

用到的飞书开放能力

整套系统没有写一行"机器人框架"，全部是开放平台原生能力的组合，按用量排序：

能力	用途
长连接事件 im.message.receive_v1	收私聊/群消息，走 WebSocket 出站连接，不需要公网回调地址——这是家庭服务器（NAT 后面）能跑起来的关键
卡片回调 card.action.trigger	审批按钮 [☒通过/记住/☒拒绝/☒取消] 的点击事件，同样走长连接
交互卡片（卡片 JSON 2.0）	任务结果卡、审批卡、进度卡；按钮 + Markdown 混排
POST /im/v1/chats	按任务自动建群，群名带状态 emoji 和短 id
PUT /im/v1/chats/:chat_id	群描述写入任务目标（进群就能看到任务是什么），执行中同步改名
DELETE /im/v1/chats/:chat_id	/done 结算后解散群，不留尸体
GET /im/v1/chats/:chat_id/members	成员核验：确认机器人和你都在群里
POST /im/v1/messages	消息发送；text 与富文本 post 双格式受理/回复

事件订阅只有两项，权限按需最小化申请即可。我用 [lark-cli](https://github.com/larksuite/cli)（官方 CLI，自带长连接消费者）做事件入口和 API 出口，appId/appSecret
由它托管，业务代码全程碰不到应用凭证。

几个较真的工程细节

1. 绝不盲目重放。 这是做 agent 自动化最容易翻车的地方。turn 请求一旦写出（哪怕响应超时、结果未知），就不允许自动重发——副作用操作重放一次可能就是灾难。重启后先对账：active 项标记 interrupted 等人确认，waiting 项保留续跑。所有受理与去重在同一个持久事务里提交，写盘失败回滚内存视图，杜绝"内存已受理/磁盘没存上"的分裂。

2. 审批无超时 + 持久重试。 审批卡片发出后，点击回调校验操作人 open_id（不是 owner 点了没用）；卡片投递失败进死信队列按退避重试；你三小时后点按钮依然有效。文字 y/ys/n/c 可以降级操作。

3. 私聊和任务群是两个世界。 私聊是你的 AI 助手（人格可用 persona 文件自定义），任务群里是纯执行单元，注入的指令明确"你不是私聊助手，不使用私聊人格"。两个命名空间的会话注册表物理分开，任务通知永不进私聊。

4. 凭证物理隔离。 私聊的 Codex 实例跑在 bubblewrap 沙箱里（nobody UID + home tmpfs），根本看不到 bridge 的飞书凭证文件；派生任务的环境变量经 sanitizeEnv 剥离所有 LARK_*/FEISHU_*；持久化和通知边界对密钥模式自动 [REDACTED]。

5. 诚实的失败。 模型过载/限流、连接中断，结果卡上如实说明发生了什么，并给出恢复路径（群内回复"继续"即可，会话线程还在）。开发期间我自己踩出一个富文本消息被静默丢弃的 bug——修复后给受理路径补了五项验收测试。

配置与部署

全部部署配置环境驱动（bridge.env 一个文件），安装是交互式的：

```
git clone https://github.com/OldManWalk/lark-codex-tasks.git
cd lark-codex-tasks && ./install.sh init
```

会引导你填 owner open_id、生成 projects.json 工作区注册表、写 systemd user 服务并启动。前置依赖：Linux + systemd、Node ≥18、登录好的 Codex CLI、配置好长连接事件的飞书自建应用（仓库里有逐步指引：docs/feishu-app-setup.md）。

安全上默认
owner-only：只有你一个人能指挥机器人。执行分三档：auto（默认，日常直接执行、特殊操作问你）/ safe（能问都问）/ yolo（全自动，建议先观望再用）。

测试

137 项验收测试覆盖：会话隔离、队列幂等、重启对账、审批生命周期、死信管理、富文本受理、密钥脱敏、模型故障路径。node --test *.test.cjs 全绿是我每次部署的门槛。

致谢与说明

本项目为独立实现，设计上受到 zarazhangrui/lark-coding-agent-bridge、xile611/lark-codex-bridge、VicLuoV5/lark-agents-bridge（均 MIT）等社区项目的启发，在此致谢。飞书/Lark 是 ByteDance 商标，Codex/OpenAI 是 OpenAI 商标，本项目与官方无关。

仓库：https://github.com/OldManWalk/lark-codex-tasks（MIT，欢迎 issue 和 PR）

如果你也在家庭服务器上跑 agent，欢迎试试——先从 safe 模式开始，建立起信任再放权。